

LAPORAN PRAKTIKUM STRUKTUR DATA

GRAPH

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat, hidayah, dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum Struktur Data dengan topik implementasi Graf (Graph) ini dengan baik dan tepat pada waktunya. Laporan ini disusun sebagai salah satu syarat akademis dalam memenuhi tugas mata kuliah Praktikum Struktur Data pada Departemen Informatika, Fakultas Teknologi Informasi, Universitas Andalas. Melalui praktikum ini, penulis telah mendapatkan banyak wawasan, pemahaman mendalam, serta pengalaman praktis mengenai representasi, penelusuran, dan penerapan struktur data graf dalam menyelesaikan permasalahan optimasi rute berbasis Graphical User Interface (GUI).

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi sistematis penulisan, kedalaman materi, maupun penyajian kode program. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat nyata bagi perkembangan ilmu pengetahuan, khususnya di bidang informatika.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 Node_2511531017.....	2
2.2 BTree_2511531017.....	5
2.3 BtreeDriver_2511531017.....	8
2.4 GraphTraversal_2511531017.....	11
TUGAS.....	15
2.5 PetaFTIUnand_2511531017.....	15
BAB III KESIMPULAN.....	28
3.1 Kesimpulan.....	28
3.2 Saran.....	28
DAFTAR PUSTAKA.....	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia informatika, representasi data yang mencerminkan hubungan kompleks antar-objek di dunia nyata sangatlah krusial. Struktur data linier atau hierarkis seperti array dan tree memiliki keterbatasan dalam memodelkan interkoneksi bebas yang bersifat many-to-many. Oleh karena itu, struktur data Graf (Graph) hadir sebagai model matematis yang sangat fleksibel dan efisien untuk merepresentasikan hubungan non-linier tersebut. Berdasarkan praktikum yang telah dilaksanakan di Universitas Andalas, teori graf ini diimplementasikan untuk memetakan hubungan keterhubungan dan optimasi rute antar-lokasi penting di lingkungan akademik, seperti rektorat, perpustakaan, kelas, dan laboratorium, di mana setiap tempat bertindak sebagai node/vertex dan rute penghubungnya sebagai edge berbobot.

Untuk menjembatani konsep graf abstrak tersebut agar lebih mudah dipahami, penggunaan antarmuka berbasis grafis atau Graphical User Interface (GUI) menjadi sebuah kebutuhan utama. Melalui pemrograman Java dengan memanfaatkan library Swing (JFrame dan JPanel) serta penanganan multi-threading, visualisasi struktur graf dapat ditampilkan secara dinamis dan interaktif. Integrasi antara algoritma penelusuran graf (seperti pelacakan jalur dan pewarnaan komponen aktif) dengan visualisasi GUI ini mempermudah pengguna dalam mengamati perubahan status data secara langsung. Oleh karena itu, praktikum ini dilakukan untuk memberikan pemahaman mendalam mengenai teknik perancangan, pengodean, penyelarasan data lewat fungsi reset, serta visualisasi struktur data graf secara nyata.

1.2 Tujuan

1. Memahami konsep dasar, terminologi, dan representasi struktural dari Graf (Graph) menggunakan bahasa pemrograman Java.
2. Mengimplementasikan penelusuran graf dan algoritma manipulasi komponennya ke dalam program aplikasi interaktif berbasis Graphical User Interface (GUI).
3. Menganalisis fungsionalitas metode penelusuran serta mengintegrasikannya dengan manajemen komponen grafis dalam multi-threading Java.

1.3 Manfaat

1. Meningkatkan kemampuan logika pemrograman dan keterampilan praktis dalam membangun aplikasi Java berbasis GUI yang menerapkan struktur data non-linier.
2. Berfungsi sebagai referensi akademis yang valid dalam memahami teknik visualisasi struktur data abstrak untuk mata kuliah lanjutan.
3. Menyediakan gambaran dasar mengenai cara kerja sistem navigasi dan optimasi rute perjalanan yang dapat diterapkan di dunia industri nyata.

BAB II

PEMBAHASAN

2.1 Node_2511531017

```
package pekan9_2511531017;

public class Node_2511531017 {
    int data_1017;
    Node_2511531017 left_1017;
    Node_2511531017 right_1017;
    public Node_2511531017(int data_1017) {
        this.data_1017 = data_1017;
        left_1017 = null;
        right_1017 = null;
    }
    public void setLeft_1017(Node_2511531017 node_1017) {
        if (left_1017 == null)
            left_1017 = node_1017;
    }
    public void setRight_1017 (Node_2511531017 node_1017) {
        if (right_1017 == null)
            right_1017 = node_1017;
    }
    public Node_2511531017 getLeft_1017() {
        return left_1017;
    }
    public Node_2511531017 getRight_1017() {
        return right_1017;
    }
    public int getData_1017 () {
        return data_1017;
    }
    public void setData_1017 (int data_1017) {
        this.data_1017 = data_1017;
    }
    void printPreorder_1017(Node_2511531017 node_1017) {
        if (node_1017 == null)
            return;
        System.out.print(node_1017.data_1017 + " ");
        printPreorder_1017(node_1017.left_1017);
        printPreorder_1017(node_1017.right_1017);
    }
    void printPostorder_1017(Node_2511531017 node_1017) {
        if (node_1017 == null)
            return;
        printPostorder_1017(node_1017.left_1017);
        printPostorder_1017(node_1017.right_1017);
        System.out.print(node_1017.data_1017 + " ");
    }
    void printInorder_1017(Node_2511531017 node_1017) {
        if (node_1017 == null)
            return;
        printInorder_1017(node_1017.left_1017);
        System.out.print(node_1017.data_1017 + " ");
    }
}
```

```

        printInorder_1017(node_1017.right_1017);
    }
    public String print_1017() {
        return this.print_1017("",true,"");
    }
    public String print_1017(String prefix_1017, boolean isTail_1017,
String sb_1017) {
        if (right_1017 != null) {
            right_1017.print_1017(prefix_1017 + (isTail_1017 ? "| " :
"  "),false,sb_1017);
        }
        System.out.println(prefix_1017 + (isTail_1017 ? "\\--" : "/--")
+ data_1017);
        if (left_1017 != null) {
            left_1017.print_1017(prefix_1017 + (isTail_1017 ? "  " :
"| "),true,sb_1017);
        }
        return sb_1017;
    }
}

```

Kode 2.1

Penjelasan:

```

public class Node_2511531017 {
    int data_1017;
    Node_2511531017 left_1017;
    Node_2511531017 right_1017;
}

```

Class ini digunakan untuk merepresentasikan satu node pada Binary Tree. Setiap node memiliki data,kiri, dan kanan. Node-node inilah yang membentuk struktur pohon.

```

public Node_2511531017(int data_1017) {
    this.data_1017 = data_1017;
    left_1017 = null;
    right_1017 = null;
}

```

Konstruktor ini digunakan untuk membuat node baru. Data node diisi sesuai parameter yang diberikan. Anak kiri dan anak kanan diinisialisasi dengan nilai null.

```

public void setLeft_1017(Node_2511531017 node_1017) {
    if (left_1017 == null)
        left_1017 = node_1017;
}

```

Untuk menambahkan node anak kiri. Penambahan hanya dilakukan jika posisi kiri masih kosong. Dengan demikian node yang sudah ada tidak akan tertimpa.

```

public void setRight_1017 (Node_2511531017 node_1017) {
    if (right_1017 == null)
        right_1017 = node_1017;
}

```

Untuk menambahkan node anak kanan. Node baru hanya ditempatkan jika posisi kanan belum terisi. Method ini membantu membangun struktur tree.

```

public Node_2511531017 getLeft_1017() {
    return left_1017;
}
public Node_2511531017 getRight_1017() {
    return right_1017;
}

```

```

    }
    public int getData_1017 () {
        return data_1017;
    }

```

Method getter digunakan untuk mengambil data atau referensi node tertentu. Getter kiri mengambil anak kiri, getter kanan mengambil anak kanan, dan getter data mengambil nilai node. Method ini mempermudah akses terhadap atribut node.

```

public void setData_1017 (int data_1017) {
    this.data_1017 = data_1017;
}

```

Digunakan untuk mengubah nilai data yang tersimpan pada node. Nilai baru diberikan melalui parameter method. Method ini termasuk ke dalam setter.

```

void printPreorder_1017(Node_2511531017 node_1017) {
    if (node_1017 == null)
        return;
    System.out.print(node_1017.data_1017 + " ");
    printPreorder_1017(node_1017.left_1017);
    printPreorder_1017(node_1017.right_1017);
}

```

Method ini melakukan traversal Preorder pada Binary Tree. Node root dicetak terlebih dahulu sebelum mengunjungi subtree kiri dan kanan. Traversal dilakukan secara rekursif.

```

void printPostorder_1017(Node_2511531017 node_1017) {
    if (node_1017 == null)
        return;
    printPostorder_1017(node_1017.left_1017);
    printPostorder_1017(node_1017.right_1017);
    System.out.print(node_1017.data_1017 + " ");
}

```

Method ini melakukan traversal Postorder. Subtree kiri dan kanan diproses terlebih dahulu sebelum root dicetak. Traversal dilakukan menggunakan rekursi.

```

void printInorder_1017(Node_2511531017 node_1017) {
    if (node_1017 == null)
        return;
    printInorder_1017(node_1017.left_1017);
    System.out.print(node_1017.data_1017 + " ");
    printInorder_1017(node_1017.right_1017);
}

```

Method ini melakukan traversal Inorder pada Binary Tree. Node kiri dicetak terlebih dahulu, kemudian root, lalu node kanan. Traversal ini sering digunakan untuk menghasilkan data terurut pada Binary Search Tree.

```

public String print_1017() {
    return this.print_1017("", true, "");
}

```

Pemanggil awal untuk mencetak struktur tree. Parameter awal diberikan secara otomatis sehingga pengguna tidak perlu mengisinya sendiri. Method ini akan memanggil versi rekursifnya.

```

public String print_1017(String prefix_1017, boolean isTail_1017, String sb_1017) {
    if (right_1017 != null) {
        right_1017.print_1017(prefix_1017 + (isTail_1017 ? "|" : " : "
        "  "), false, sb_1017);
    }
}

```

```

        System.out.println(prefix_1017 + (isTail_1017 ? "\\--" : "/--")
+ data_1017);
        if (left_1017 != null) {
            left_1017.print_1017(prefix_1017 + (isTail_1017 ? " " :
"| "), true, sb_1017);
        }
        return sb_1017;

```

Digunakan untuk menampilkan struktur Binary Tree dalam bentuk visual bercabang. Setiap node dicetak dengan simbol tertentu sehingga hubungan parent dan child terlihat jelas. Proses pencetakan dilakukan secara rekursif pada subtree kanan dan kiri.

Class Node_2511531017 digunakan sebagai pembentuk utama struktur Binary Tree karena setiap node menyimpan data serta hubungan ke node anak kiri dan kanan. Class ini memungkinkan proses penyimpanan, pencarian, dan penelusuran data dilakukan secara terstruktur dalam bentuk pohon. Dengan adanya class Node, operasi traversal seperti Preorder, Inorder, dan Postorder dapat dijalankan untuk mengakses data pada Binary Tree.

2.2 BTree_2511531017

```

package pekan9_2511531017;

public class BTree_2511531017 {
    private Node_2511531017 root_1017;
    private Node_2511531017 currentNode_1017;
    public BTree_2511531017() {
        root_1017 = null;
    }
    public boolean search_1017(int data_1017) {
        return search_1017(root_1017, data_1017);
    }
    private boolean search_1017(Node_2511531017 node_1017, int data_1017) {
        if (node_1017.getData_1017() == data_1017)
            return true;
        if (node_1017.getRight_1017() != null)
            if (search_1017(node_1017.getLeft_1017(), data_1017))
                return true;
        return false;
    }
    public void printInoreder_1017() {
        root_1017.printInorder_1017(root_1017);
    }
    public void printPreoreder_1017() {
        root_1017.printPreorder_1017(root_1017);
    }
    public void printPostoreder_1017() {
        root_1017.printPostorder_1017(root_1017);
    }
    public Node_2511531017 getRoot_1017() {
        return root_1017;
    }
    public boolean isEmpty_1017() {
        return root_1017 == null;
    }
}

```



```

    }
    public int countNodes_1017() {
        return countNodes_1017(root_1017);
    }
    private int countNodes_1017(Node_2511531017 node_1017) {
        int count_1017 = 1;
        if (node_1017== null) {
            return 0;
        }else {
            count_1017 += countNodes_1017(node_1017.getLeft_1017());
            count_1017 += countNodes_1017(node_1017.getRight_1017());
            return count_1017;
        }
    }
    public void print_1017() {
        root_1017.print_1017();
    }
    public Node_2511531017 getCurrent() {
        return currentNode_1017;
    }
    public void setCurrent(Node_2511531017 node_1017) {
        this.currentNode_1017 = node_1017;
    }
    public void setRoot_1017 (Node_2511531017 root_1017) {
        this.root_1017 = root_1017;
    }
}

```

Kode 2.2

Penjelasan:

```

public boolean search_1017(int data_1017) {
    return search_1017(root_1017,data_1017);
}

```

Memulai proses pencarian data pada Binary Tree. Pencarian dimulai dari node root dengan memanggil method rekursif search_1017(). Method akan mengembalikan nilai true jika data ditemukan dan false jika tidak ditemukan.

```

private boolean search_1017(Node_2511531017 node_1017, int data_1017) {
    if (node_1017.getData_1017()== data_1017)
        return true;
    if (node_1017.getRight_1017() != null)
        if (search_1017(node_1017.getLeft_1017(), data_1017))
            return true;
    return false;
}

```

Melakukan pencarian data secara rekursif pada Binary Tree. Program akan membandingkan data yang dicari dengan data yang terdapat pada node saat ini. Jika data ditemukan maka method mengembalikan nilai true, sedangkan jika tidak ditemukan maka akan mengembalikan nilai false.

```

public void printInoreder_1017() {
    root_1017.printInorder_1017(root_1017);
}
public void printPreoreder_1017() {
    root_1017.printPreorder_1017(root_1017);
}

```

```
public void printPostorder_1017() {
    root_1017.printPostorder_1017(root_1017);
}
```

Menampilkan isi Binary Tree menggunakan traversal. Traversal yang tersedia adalah Inorder, Preorder, dan Postorder. Setiap method akan memanggil traversal yang sesuai pada node root.

```
public Node_2511531017 getRoot_1017() {
    return root_1017;
}
public boolean isEmpty_1017() {
    return root_1017 == null;
}
```

Method `getRoot_1017()` digunakan untuk mengambil node root dari Binary Tree. Method `isEmpty_1017()` digunakan untuk memeriksa apakah tree masih kosong atau sudah memiliki node. Jika root bernilai null, maka tree dianggap kosong.

```
public int countNodes_1017() {
    return countNodes_1017(root_1017);
}
```

Menghitung jumlah seluruh node yang terdapat pada Binary Tree. Perhitungan dimulai dari root dan dilakukan secara rekursif. Hasil akhirnya berupa jumlah total node yang ada dalam tree.

```
private int countNodes_1017(Node_2511531017 node_1017) {
    int count_1017 = 1;
    if (node_1017 == null) {
        return 0;
    } else {
        count_1017 += countNodes_1017(node_1017.getLeft_1017());
        count_1017 += countNodes_1017(node_1017.getRight_1017());
        return count_1017;
    }
}
```

Menghitung jumlah node secara rekursif. Program akan menghitung node saat ini, kemudian menambahkan jumlah node pada subtree kiri dan subtree kanan. Proses berlangsung hingga seluruh node dalam tree selesai dihitung.

```
public void print_1017() {
    root_1017.print_1017();
}
```

Menampilkan struktur atau isi Binary Tree.

```
public Node_2511531017 getCurrent() {
    return currentNode_1017;
}
public void setCurrent(Node_2511531017 node_1017) {
    this.currentNode_1017 = node_1017;
}
```

Method `getCurrent()` digunakan untuk mengambil node yang sedang aktif atau sedang diproses. Method `setCurrent()` digunakan untuk mengubah nilai node yang sedang aktif. Kedua method ini membantu dalam pengelolaan posisi node saat operasi tree dilakukan.

```
public void setRoot_1017 (Node_2511531017 root_1017) {
    this.root_1017 = root_1017;
}
```

Mengubah atau menetapkan node root pada Binary Tree. Nilai root yang baru akan menggantikan root sebelumnya. Method ini biasanya digunakan saat proses pembentukan atau pengubahan struktur tree.

Class BTree_2511531017 digunakan untuk mengelola struktur Binary Tree. Class ini menyediakan berbagai operasi seperti pencarian data, traversal (Inorder, Preorder, dan Postorder), pengecekan tree kosong, perhitungan jumlah node, serta pengaturan node root dan node aktif. Seluruh operasi dilakukan dengan memanfaatkan objek dari class Node_2511531017.

2.3 BtreeDriver_2511531017

```
package pekan9_2511531017;

public class BtreeDriver_2511531017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        BTree_2511531017 tree_1017 = new BTree_2511531017();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_1017.countNodes_1017());

        Node_2511531017 root_1017 = new Node_2511531017(1);

        tree_1017.setRoot_1017(root_1017);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_1017.countNodes_1017());
        Node_2511531017 node2_1017 = new Node_2511531017(2);
        Node_2511531017 node3_1017 = new Node_2511531017(3);
        Node_2511531017 node4_1017 = new Node_2511531017(4);
        Node_2511531017 node5_1017 = new Node_2511531017(5);
        Node_2511531017 node6_1017 = new Node_2511531017(6);
        Node_2511531017 node7_1017 = new Node_2511531017(7);
        Node_2511531017 node8_1017 = new Node_2511531017(8);
        Node_2511531017 node9_1017 = new Node_2511531017(9);
        root_1017.setLeft_1017(node2_1017);
        node2_1017.setLeft_1017(node4_1017);
        node2_1017.setRight_1017(node5_1017);
        node4_1017.setRight_1017(node8_1017);
        root_1017.setRight_1017(node3_1017);
        node3_1017.setLeft_1017(node6_1017);
        node3_1017.setRight_1017(node7_1017);
        node6_1017.setLeft_1017(node9_1017);

        tree_1017.setCurrent(tree_1017.getRoot_1017());
        System.out.println("menampilkan simpul terakhir");
        System.out.println(tree_1017.getCurrent().getData_1017());
        System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
        System.out.println(tree_1017.countNodes_1017());
        System.out.println("InOrder: ");
        tree_1017.printInoreder_1017();
        System.out.println("\nPreOrder: ");
        tree_1017.printPreoreder_1017();
        System.out.println("\nPostOrder: ");
        tree_1017.printPostoreder_1017();
        System.out.println("\nmenampilkan simpuk dalam bentuk pohon");
        tree_1017.print_1017();
    }
}
```

```

    }
}

```

Kode 2.3

Penjelasan:

```

public static void main(String[] args) {
    // TODO Auto-generated method stub
    BTree_2511531017 tree_1017 = new BTree_2511531017();
    System.out.print("Jumlah Simpul awal pohon: ");
    System.out.println(tree_1017.countNodes_1017());
}

```

Digunakan untuk membuat objek Binary Tree baru yang masih kosong. Setelah tree dibuat, program menampilkan jumlah simpul yang ada pada tree menggunakan method `countNodes_1017()`. Karena belum ada node yang ditambahkan, jumlah simpul yang ditampilkan adalah 0.

```

Node_2511531017 root_1017 = new Node_2511531017(1);

tree_1017.setRoot_1017(root_1017);
System.out.println("Jumlah simpul jika hanya ada root");
System.out.println(tree_1017.countNodes_1017());

```

Membuat node root dengan nilai 1. Node tersebut kemudian dijadikan akar dari Binary Tree menggunakan method `setRoot_1017()`. Setelah root ditambahkan, jumlah simpul pada tree menjadi 1.

```

Node_2511531017 node2_1017 = new Node_2511531017(2);
Node_2511531017 node3_1017 = new Node_2511531017(3);
Node_2511531017 node4_1017 = new Node_2511531017(4);
Node_2511531017 node5_1017 = new Node_2511531017(5);
Node_2511531017 node6_1017 = new Node_2511531017(6);
Node_2511531017 node7_1017 = new Node_2511531017(7);
Node_2511531017 node8_1017 = new Node_2511531017(8);
Node_2511531017 node9_1017 = new Node_2511531017(9);

```

Membuat beberapa node yang akan dimasukkan ke dalam Binary Tree. Setiap node memiliki nilai data yang berbeda mulai dari 2 hingga 9. Node node tersebut nantinya akan dihubungkan sehingga membentuk struktur pohon biner.

```

root_1017.setLeft_1017(node2_1017);
node2_1017.setLeft_1017(node4_1017);
node2_1017.setRight_1017(node5_1017);
node4_1017.setRight_1017(node8_1017);
root_1017.setRight_1017(node3_1017);
node3_1017.setLeft_1017(node6_1017);
node3_1017.setRight_1017(node7_1017);
node6_1017.setLeft_1017(node9_1017);

```

Menghubungkan node-node yang telah dibuat sehingga membentuk Binary Tree. Method `setLeft_1017()` digunakan untuk menambahkan anak kiri, sedangkan `setRight_1017()` digunakan untuk menambahkan anak kanan. Setelah seluruh node terhubung, terbentuklah struktur pohon yang lengkap.

```

tree_1017.setCurrent(tree_1017.getRoot_1017());
System.out.println("menampilkan simpul terakhir");

```

```
System.out.println(tree_1017.getCurrent().getData_1017());
```

Untuk mengatur node yang sedang aktif menjadi root tree. Method `getRoot_1017()` mengambil node akar, kemudian disimpan ke variabel `current` menggunakan `setCurrent()`. Setelah itu data dari node `current` ditampilkan ke layar.

```
System.out.println("Jumlah simpul; setelah simpul 7  
ditambahkan");  
System.out.println(tree_1017.countNodes_1017());
```

Menghitung jumlah seluruh simpul yang terdapat pada Binary Tree. Perhitungan dilakukan menggunakan method `countNodes_1017()` yang bekerja secara rekursif. Hasil yang ditampilkan menunjukkan total node yang sudah terhubung pada tree.

```
System.out.println("InOrder: ");  
tree_1017.printInoreder_1017();
```

Menampilkan traversal Inorder pada Binary Tree. Traversal dilakukan dengan urutan kiri, root, kemudian kanan. Hasil traversal akan dicetak ke layar sesuai urutan tersebut.

```
System.out.println("\nPreOrder: ");  
tree_1017.printPreoreder_1017();
```

Menampilkan traversal Preorder pada Binary Tree. Traversal dilakukan dengan urutan root, kiri, kemudian kanan. Hasil traversal akan menunjukkan urutan kunjungan node dari akar terlebih dahulu.

```
System.out.println("\nPostOrder: ");  
tree_1017.printPostoreder_1017();
```

Menampilkan traversal Postorder pada Binary Tree. Traversal dilakukan dengan urutan kiri, kanan, kemudian root. Hasil traversal dicetak ke layar sesuai urutan kunjungan tersebut.

```
System.out.println("\nmenampilkan simpuk dalam bentuk pohon");  
tree_1017.print_1017();
```

Menampilkan struktur Binary Tree dalam bentuk visual menyerupai pohon. Setiap node akan dicetak sesuai posisi `parent` dan `child` yang dimilikinya. Tampilan ini memudahkan pengguna untuk melihat struktur tree secara keseluruhan.

Output:

```
<terminated> BtreeDriver_2511531017 [Java Application] C:\Users\SISI\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\ja
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
menampilkan simpuk dalam bentuk pohon
|
|--7
|
|--3
|   |--6
|   |--9
|--1
|   |--5
|   |--2
|       |--8
|       |--4
```

Gambar 2.3

2.4 GraphTraversal_2511531017

```
package pekan9_2511531017;
import java.util.*;
public class GraphTraversal_2511531017 {
    private Map<String, List<String>> graph_1017 = new HashMap<>();

    public void addEdge_1017 (String node1_1017, String node2_1017) {
        graph_1017.putIfAbsent(node1_1017, new ArrayList<>());
        graph_1017.putIfAbsent(node2_1017, new ArrayList<>());
        graph_1017.get(node1_1017).add(node2_1017);
        graph_1017.get(node2_1017).add(node1_1017);
    }

    public void printGraph_1017() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_1017 : graph_1017.keySet()) {
            System.out.print(node_1017 + " -> ");
            List<String> neighbors_1017 = graph_1017.get(node_1017);
            System.out.println(String.join(", ", neighbors_1017));
        }
        System.out.println();
    }

    public void dfs_1017(String start_1017) {
        Set<String> visited_1017 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper_1017 (start_1017, visited_1017);
        System.out.println();
    }

    private void dfsHelper_1017(String current_1017, Set<String>
visited_1017) {
        if (visited_1017.contains(current_1017)) return;
        visited_1017.add(current_1017);
        System.out.print(current_1017 + " ");
        for (String neighbor_1017 : graph_1017.getDefault(current_1017,
new ArrayList<>())) {
            dfsHelper_1017(neighbor_1017, visited_1017);
        }
    }

    public void bfs_1017(String start_1017) {
        Set<String> visited_1017 = new HashSet<>();
        Queue<String> queue_1017 = new LinkedList<>();
        queue_1017.add(start_1017);
        visited_1017.add(start_1017);
        System.out.println("Penelusuran BFS:");
        while (!queue_1017.isEmpty()) {
            String current_1017 = queue_1017.poll();
            System.out.print(current_1017 + " ");
            for (String neighbor_1017 :
graph_1017.getDefault(current_1017, new ArrayList<>())) {
                if (!visited_1017.contains(neighbor_1017)) {
                    queue_1017.add(neighbor_1017);
                    visited_1017.add(neighbor_1017);
                }
            }
        }
        System.out.println();
    }
}
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        GraphTraversal_2511531017 graph_1017 = new
GraphTraversal_2511531017();

        graph_1017.addEdge_1017("A", "B");
        graph_1017.addEdge_1017("A", "C");
        graph_1017.addEdge_1017("B", "D");
        graph_1017.addEdge_1017("B", "E");
        System.out.println("Graf Awal adalah: ");
        graph_1017.printGraph_1017();
        graph_1017.dfs_1017("A");
        graph_1017.bfs_1017("B");

    }
}

```

Kode 2.4

Penjelasan:

```

private Map<String, List<String>> graph_1017 = new HashMap<>();

```

Untuk membuat struktur data graf menggunakan Map dan List.

```

public void addEdge_1017 (String node1_1017, String node2_1017) {
    graph_1017.putIfAbsent(node1_1017, new ArrayList<>());
    graph_1017.putIfAbsent(node2_1017, new ArrayList<>());
    graph_1017.get(node1_1017).add(node2_1017);
    graph_1017.get(node2_1017).add(node1_1017);
}

```

Menambahkan sisi antara dua simpul pada graf. Perintah putIfAbsent() memastikan bahwa simpul sudah tersedia di dalam graf sebelum hubungan ditambahkan. Karena graf bersifat tidak berarah (undirected graph), maka hubungan ditambahkan ke kedua arah.

```

public void printGraph_1017() {
    System.out.println("Graf Awal (Adjacency List):");
    for (String node_1017 : graph_1017.keySet()) {
        System.out.print(node_1017 + " -> ");
        List<String> neighbors_1017 = graph_1017.get(node_1017);
        System.out.println(String.join(", ", neighbors_1017));
    }
    System.out.println();
}

```

Menampilkan isi graf dalam bentuk adjacency list. Setiap simpul akan ditampilkan beserta daftar simpul tetangganya. Tampilan ini memudahkan pengguna untuk melihat struktur hubungan yang terbentuk pada graf.

```

public void dfs_1017(String start_1017) {
    Set<String> visited_1017 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_1017 (start_1017, visited_1017);
    System.out.println();
}

```

memulai proses penelusuran graf menggunakan algoritma Depth First Search (DFS). Sebuah HashSet dibuat untuk menyimpan simpul yang sudah dikunjungi agar tidak

terjadi pengulangan. Selanjutnya proses penelusuran dilakukan oleh method rekursif dfsHelper_1017().

```
private void dfsHelper_1017(String current_1017, Set<String> visited_1017) {  
    if (visited_1017.contains(current_1017)) return;  
    visited_1017.add(current_1017);  
    System.out.print(current_1017 + " ");  
    for (String neighbor_1017 : graph_1017.getDefault(current_1017,  
new ArrayList<>())) {  
        dfsHelper_1017(neighbor_1017, visited_1017);  
    }  
}
```

Method ini merupakan inti dari algoritma DFS yang menggunakan rekursi. Jika suatu simpul sudah pernah dikunjungi maka proses langsung dihentikan untuk menghindari perulangan. Selanjutnya simpul dicetak dan seluruh tetangganya ditelusuri secara mendalam satu per satu hingga mencapai simpul terakhir.

```
public void bfs_1017(String start_1017) {  
    Set<String> visited_1017 = new HashSet<>();  
    Queue<String> queue_1017 = new LinkedList<>();  
    queue_1017.add(start_1017);  
    visited_1017.add(start_1017);  
    System.out.println("Penelusuran BFS:");  
}
```

Bagian ini digunakan untuk menyiapkan proses Breadth First Search (BFS). Struktur data Queue digunakan untuk menyimpan simpul yang akan diproses secara berurutan. Simpul awal dimasukkan ke antrean dan langsung ditandai sebagai telah dikunjungi.

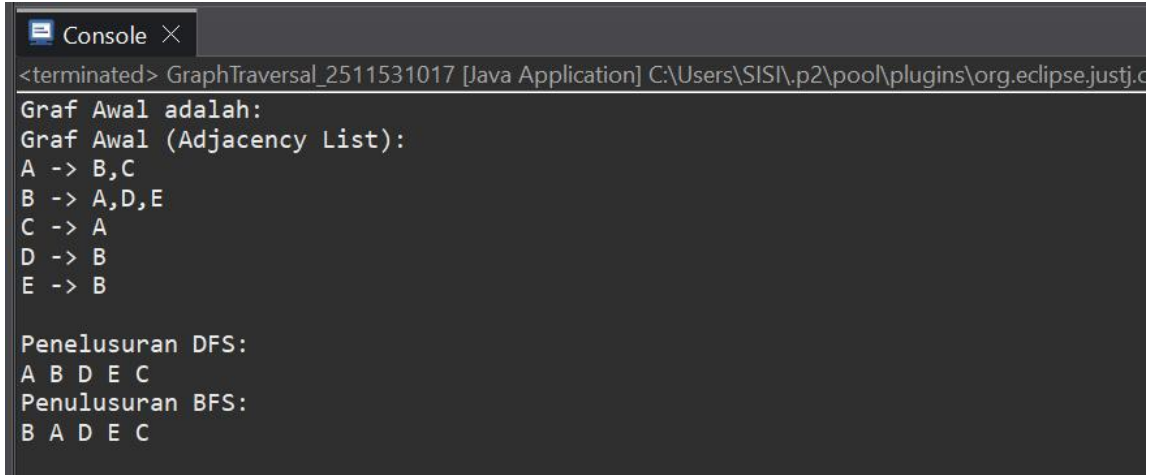
```
while (!queue_1017.isEmpty()) {  
    String current_1017 = queue_1017.poll();  
    System.out.print(current_1017 + " ");  
    for (String neighbor_1017 :  
graph_1017.getDefault(current_1017, new ArrayList<>())) {  
        if (!visited_1017.contains(neighbor_1017)) {  
            queue_1017.add(neighbor_1017);  
            visited_1017.add(neighbor_1017);  
        }  
    }  
}
```

Algoritma BFS, simpul yang berada di depan antrean akan diambil kemudian seluruh tetangganya diperiksa. Tetangga yang belum pernah dikunjungi akan dimasukkan ke antrean sehingga penelusuran dilakukan per level atau per kedalaman yang sama terlebih dahulu.

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    GraphTraversal_2511531017 graph_1017 = new  
GraphTraversal_2511531017();  
  
    graph_1017.addEdge_1017("A", "B");  
    graph_1017.addEdge_1017("A", "C");  
    graph_1017.addEdge_1017("B", "D");  
    graph_1017.addEdge_1017("B", "E");  
    System.out.println("Graf Awal adalah: ");  
    graph_1017.printGraph_1017();  
    graph_1017.dfs_1017("A");  
    graph_1017.bfs_1017("B");  
}
```

Pada bagian ini dibuat objek graf dan ditambahkan beberapa hubungan antar simpul menggunakan method addEdge_1017(). Setelah graf terbentuk, program menampilkan graf serta melakukan penelusuran menggunakan algoritma DFS dan BFS.

Output:

A screenshot of a Java console window titled "Console X". The window shows the output of a Java application. The text in the console is as follows:

```
<terminated> GraphTraversal_2511531017 [Java Application] C:\Users\SISI\p2\pool\plugins\org.eclipse.justj.c
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B,C
B -> A,D,E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
B A D E C
```

Gambar 2.4

TUGAS

2.5 PetaFTIUnand_2511531017

```
package pekan9_2511531017;
import javax.swing.*;

public class PetaFTIUnand_2511531017 extends JFrame {

    private Map<String, Point> nodePositions_1017;
    private List<String> vertices_1017;
    private Map<String, List<String>> adjacencyList_1017;

    private JPanel visualizationPanel_1017;
    private JComboBox<String> startComboBox_1017;
    private JComboBox<String> goalComboBox_1017;
    private JTextArea resultArea_1017;
    private JLabel exploredCountLabel_1017;

    private List<String> path_1017;
    private List<String> visitedOrder_1017;
    private Set<String> visitedNodes_1017;
    private String algorithmUsed_1017 = "";

    public PetaFTIUnand_2511531017() {
        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
        setSize(1100, 750);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        initializeGraph_1017();
        initializeGUI_1017();
    }

    private void initializeGraph_1017() {
        vertices_1017 = Arrays.asList("Parkiran", "Pascasarjana", "Lab Teknik Komputer", "Lab Sistem Informasi",
            "Seminar E", "PKM", "Dept Teknik Komputer", "Dept Sistem Informasi", "Lab Informatika", "Lobby",
            "Dekanat", "Mushalla", "Dept Informatika");

        nodePositions_1017 = new HashMap<>();
        nodePositions_1017.put("Parkiran", new Point(350, 80));
        nodePositions_1017.put("Pascasarjana", new Point(120, 180));
        nodePositions_1017.put("Lab Teknik Komputer", new Point(350, 180));
        nodePositions_1017.put("Lab Sistem Informasi", new Point(580, 180));
        nodePositions_1017.put("Seminar E", new Point(800, 180));
        nodePositions_1017.put("PKM", new Point(120, 320));
        nodePositions_1017.put("Dept Teknik Komputer", new Point(350, 320));
        nodePositions_1017.put("Dept Sistem Informasi", new Point(580, 320));
        nodePositions_1017.put("Lab Informatika", new Point(800, 320));
        nodePositions_1017.put("Lobby", new Point(120, 480));
        nodePositions_1017.put("Dekanat", new Point(350, 480));
        nodePositions_1017.put("Mushalla", new Point(580, 480));
        nodePositions_1017.put("Dept Informatika", new Point(800, 480));

        adjacencyList_1017 = new HashMap<>();
        for (String v_1017 : vertices_1017) {
            adjacencyList_1017.put(v_1017, new ArrayList<>());
        }

        addEdge_1017("Parkiran", "Pascasarjana");
        addEdge_1017("Parkiran", "Lab Teknik Komputer");
        addEdge_1017("Pascasarjana", "PKM");
        addEdge_1017("Lab Teknik Komputer", "Dept Teknik Komputer");
        addEdge_1017("Lab Sistem Informasi", "Seminar E");
        addEdge_1017("Lab Sistem Informasi", "Dept Sistem Informasi");
        addEdge_1017("PKM", "Lobby");
        addEdge_1017("Dept Teknik Komputer", "Dept Sistem Informasi");
        addEdge_1017("Dept Teknik Komputer", "Dekanat");
        addEdge_1017("Dept Sistem Informasi", "Lab Informatika");
        addEdge_1017("Dept Sistem Informasi", "Mushalla");
        addEdge_1017("Lab Informatika", "Dept Informatika");
        addEdge_1017("Lab Informatika", "Seminar E");
    }
}
```

```

        addEdge_1017("Lobby", "Dekanat");
        addEdge_1017("Dekanat", "Mushalla");
        addEdge_1017("Mushalla", "Dept Informatika");
    }

    private void addEdge_1017(String u_1017, String v_1017) {
        adjacencyList_1017.get(u_1017).add(v_1017);
        adjacencyList_1017.get(v_1017).add(u_1017);
    }

    private void initializeGUI_1017() {
        setLayout(new BorderLayout());

        JPanel controlPanel_1017 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));
        controlPanel_1017.setBackground(new Color(240, 240, 240));

        startComboBox_1017 = new JComboBox<>(vertices_1017.toArray(new String[0]));
        goalComboBox_1017 = new JComboBox<>(vertices_1017.toArray(new String[0]));
        JButton btnBFS_1017 = new JButton("BFS");
        JButton btnDFS_1017 = new JButton("DFS");
        JButton btnReset_1017 = new JButton("RESET");

        btnBFS_1017.setBackground(new Color(70, 130, 180));
        btnBFS_1017.setForeground(Color.WHITE);
        btnDFS_1017.setBackground(new Color(34, 139, 34));
        btnDFS_1017.setForeground(Color.WHITE);
        btnReset_1017.setBackground(new Color(220, 20, 60));
        btnReset_1017.setForeground(Color.WHITE);

        controlPanel_1017.add(new JLabel("Lokasi Awal:"));
        controlPanel_1017.add(startComboBox_1017);
        controlPanel_1017.add(new JLabel("Lokasi Tujuan:"));
        controlPanel_1017.add(goalComboBox_1017);
        controlPanel_1017.add(btnBFS_1017);

        controlPanel_1017.add(btnDFS_1017);
        controlPanel_1017.add(btnReset_1017);
        add(controlPanel_1017, BorderLayout.NORTH);

        visualizationPanel_1017 = new JPanel() {
            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                drawGraph_1017(g);
            }
        };
        visualizationPanel_1017.setBackground(Color.WHITE);
        add(visualizationPanel_1017, BorderLayout.CENTER);

        JPanel resultPanel_1017 = new JPanel(new BorderLayout());
        resultPanel_1017.setBorder(BorderFactory.createTitledBorder("Hasil Pencarian"));
        resultPanel_1017.setBackground(new Color(245, 245, 245));

        resultArea_1017 = new JTextArea();
        resultArea_1017.setEditable(false);
        resultArea_1017.setFont(new Font("Monospaced", Font.PLAIN, 13));
        resultArea_1017.setLineWrap(true);
        resultArea_1017.setWrapStyleWord(true);

        exploredCountLabel_1017 = new JLabel("Jumlah Node Dikunjungi: 0");
        exploredCountLabel_1017.setFont(new Font("Arial", Font.BOLD, 14));
        exploredCountLabel_1017.setBorder(BorderFactory.createEmptyBorder(5, 15, 5, 15));

        resultPanel_1017.add(new JScrollPane(resultArea_1017), BorderLayout.CENTER);
        resultPanel_1017.add(exploredCountLabel_1017, BorderLayout.EAST);

        resultPanel_1017.setPreferredSize(new Dimension(1000, 130));
        add(resultPanel_1017, BorderLayout.SOUTH);
    }

```

```

        btnBFS_1017.addActionListener(e -> BFS_1017());
        btnDFS_1017.addActionListener(e -> DFS_1017());
        btnReset_1017.addActionListener(e -> resetGraph_1017());
    }

    public void displayGraph_1017() {
        visualizationPanel_1017.repaint();
    }

    private void drawGraph_1017(Graphics g) {
        Graphics2D g2d_1017 = (Graphics2D) g;
        g2d_1017.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        g2d_1017.setColor(Color.LIGHT_GRAY);
        g2d_1017.setStroke(new BasicStroke(3));
        for (String u_1017 : adjacencyList_1017.keySet()) {
            Point p1_1017 = nodePositions_1017.get(u_1017);
            for (String v_1017 : adjacencyList_1017.get(u_1017)) {
                Point p2_1017 = nodePositions_1017.get(v_1017);
                g2d_1017.drawLine(p1_1017.x, p1_1017.y, p2_1017.x, p2_1017.y);
            }
        }

        if (path_1017 != null && path_1017.size() > 1) {
            g2d_1017.setColor(Color.RED);
            g2d_1017.setStroke(new BasicStroke(5));
            for (int i = 0; i < path_1017.size() - 1; i++) {
                Point p1_1017 = nodePositions_1017.get(path_1017.get(i));
                Point p2_1017 = nodePositions_1017.get(path_1017.get(i+1));
                g2d_1017.drawLine(p1_1017.x, p1_1017.y, p2_1017.x, p2_1017.y);
            }
            g2d_1017.setStroke(new BasicStroke(3));
        }
    }

    int width_1017 = 130;
    int height_1017 = 50;

    for (String node_1017 : vertices_1017) {
        Point p_1017 = nodePositions_1017.get(node_1017);
        int x_1017 = p_1017.x - width_1017 / 2;
        int y_1017 = p_1017.y - height_1017 / 2;

        if (path_1017 != null && path_1017.contains(node_1017)) {
            g2d_1017.setColor(new Color(255, 100, 100));
        } else if (visitedNodes_1017 != null && visitedNodes_1017.contains(node_1017)) {
            g2d_1017.setColor(new Color(255, 220, 100));
        } else {
            g2d_1017.setColor(new Color(173, 216, 230));
        }

        g2d_1017.fillRoundRect(x_1017, y_1017, width_1017, height_1017, 10, 10);
        g2d_1017.setColor(Color.BLACK);
        g2d_1017.setStroke(new BasicStroke(2));
        g2d_1017.drawRoundRect(x_1017, y_1017, width_1017, height_1017, 10, 10);

        // Teks nama node
        FontMetrics fm_1017 = g2d_1017.getFontMetrics();
        int textWidth_1017 = fm_1017.stringWidth(node_1017);
        g2d_1017.drawString(node_1017, p_1017.x - textWidth_1017 / 2, p_1017.y + 5);
    }
}

public void BFS_1017() {
    String start_1017 = (String) startComboBox_1017.getSelectedItem();
    String goal_1017 = (String) goalComboBox_1017.getSelectedItem();

    if (start_1017.equals(goal_1017)) {
        resultArea_1017.setText("Lokasi awal dan tujuan sama!");
    }
}

```



```

        return;
    }

    Queue<String> queue_1017 = new LinkedList<>();
    Map<String, String> parent_1017 = new HashMap<>();
    Set<String> visited_1017 = new LinkedHashSet<>();

    queue_1017.add(start_1017);
    visited_1017.add(start_1017);
    parent_1017.put(start_1017, null);

    boolean found_1017 = false;

    while(!queue_1017.isEmpty()) {
        String current_1017 = queue_1017.poll();
        if(current_1017.equals(goal_1017)) {
            found_1017 = true;
            break;
        }

        for(String neighbor_1017 : adjacencyList_1017.get(current_1017)) {
            if(!visited_1017.contains(neighbor_1017)) {
                visited_1017.add(neighbor_1017);
                parent_1017.put(neighbor_1017, current_1017);
                queue_1017.add(neighbor_1017);
            }
        }
    }

    visitedOrder_1017 = new ArrayList<>(visited_1017);
    visitedNodes_1017 = visited_1017;
    exploredCountLabel_1017.setText("Jumlah Node Dikunjungi: " + visited_1017.size());
    algorithmUsed_1017 = "BFS";

    if(found_1017) {
        path_1017 = new ArrayList<>();
        String curr_1017 = goal_1017;
        while(curr_1017 != null) {
            path_1017.add(curr_1017);
            curr_1017 = parent_1017.get(curr_1017);
        }
        Collections.reverse(path_1017);
        displayPath_1017();
    } else {
        resultArea_1017.setText("Jalur tidak ditemukan!");
        path_1017 = null;
    }
    displayGraph_1017();
}

public void DFS_1017() {
    String start_1017 = (String) startComboBox_1017.getSelectedItem();
    String goal_1017 = (String) goalComboBox_1017.getSelectedItem();

    if (start_1017.equals(goal_1017)) {
        resultArea_1017.setText("Lokasi awal dan tujuan sama!");
        return;
    }

    Stack<String> stack_1017 = new Stack<>();
    Map<String, String> parent_1017 = new HashMap<>();
    Set<String> visited_1017 = new LinkedHashSet<>();

    stack_1017.push(start_1017);
    visited_1017.add(start_1017);
    parent_1017.put(start_1017, null);

    boolean found_1017 = false;

```

```

        while(!stack_1017.isEmpty()) {
            String current_1017 = stack_1017.pop();
            if(current_1017.equals(goal_1017)) {
                found_1017 = true;
                break;
            }

            for(String neighbor_1017 : adjacencyList_1017.get(current_1017)) {
                if(!visited_1017.contains(neighbor_1017)) {
                    visited_1017.add(neighbor_1017);
                    parent_1017.put(neighbor_1017, current_1017);
                    stack_1017.push(neighbor_1017);
                }
            }
        }

        visitedOrder_1017 = new ArrayList<>(visited_1017);
        visitedNodes_1017 = visited_1017;
        exploredCountLabel_1017.setText("Jumlah Node Dikunjungi: " + visited_1017.size());
        algorithmUsed_1017 = "DFS";

        if(found_1017) {
            path_1017 = new ArrayList<>();
            String curr_1017 = goal_1017;
            while(curr_1017 != null) {
                path_1017.add(curr_1017);
                curr_1017 = parent_1017.get(curr_1017);
            }
            Collections.reverse(path_1017);
            displayPath_1017();
        } else {
            resultArea_1017.setText("Jalur tidak ditemukan!");
            path_1017 = null;
        }
    }

    displayGraph_1017();
}

public void displayPath_1017() {
    StringBuilder sb_1017 = new StringBuilder();
    sb_1017.append("Algoritma: ").append(algorithmUsed_1017).append("\n\n");
    sb_1017.append("Jalur:\n");
    if (path_1017 != null) {
        sb_1017.append(String.join(" -> ", path_1017)).append("\n\n");
    }
    sb_1017.append("Node Dikunjungi:\n");
    if (visitedOrder_1017 != null) {
        sb_1017.append(String.join(", ", visitedOrder_1017));
    }
    resultArea_1017.setText(sb_1017.toString());
}

public void resetGraph_1017() {
    path_1017 = null;
    visitedOrder_1017 = null;
    visitedNodes_1017 = null;
    algorithmUsed_1017 = "";
    exploredCountLabel_1017.setText("Jumlah Node Dikunjungi: 0");
    resultArea_1017.setText("");
    displayGraph_1017();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new PetaFTIUnand_2511531017().setVisible(true);
    });
}
}

```

Kode 2.5

Penjelasan:

```

public PetaFTIUnand_2511531017() {
    setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
    setSize(1100, 750);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);

    initializeGraph_1017();
    initializeGUI_1017();
}

```

Konstruktor digunakan untuk membuat dan mengatur jendela utama aplikasi. Pada method ini ditentukan judul program, ukuran jendela, serta aksi ketika tombol close ditekan. Selain itu method ini juga memanggil proses pembuatan graf dan tampilan GUI.

```
private void initializeGraph_1017() {
    vertices_1017 = Arrays.asList("Parkiran", "Pascasarjana", "Lab Teknik
Komputer", "Lab Sistem Informasi",
    "Seminar E", "PKM", "Dept Teknik Komputer", "Dept Sistem
Informasi", "Lab Informatika", "Lobby",
    "Dekanat", "Mushalla", "Dept Informatika");
```

Membuat daftar node (vertex) yang merepresentasikan lokasi lokasi pada peta FTI UNAND.

```
nodePositions_1017 = new HashMap<>();
nodePositions_1017.put("Parkiran", new Point(350, 80));
nodePositions_1017.put("Pascasarjana", new Point(120, 180));
nodePositions_1017.put("Lab Teknik Komputer", new Point(350, 180));
nodePositions_1017.put("Lab Sistem Informasi", new Point(580, 180));
nodePositions_1017.put("Seminar E", new Point(800, 180));
nodePositions_1017.put("PKM", new Point(120, 320));
nodePositions_1017.put("Dept Teknik Komputer", new Point(350, 320));
nodePositions_1017.put("Dept Sistem Informasi", new Point(580, 320));
nodePositions_1017.put("Lab Informatika", new Point(800, 320));
nodePositions_1017.put("Lobby", new Point(120, 480));
nodePositions_1017.put("Dekanat", new Point(350, 480));
nodePositions_1017.put("Mushalla", new Point(580, 480));
nodePositions_1017.put("Dept Informatika", new Point(800, 480));
```

Kode ini digunakan untuk menentukan posisi setiap node pada tampilan GUI agar graf dapat divisualisasikan sesuai layout tertentu menggunakan koordinat x dan y.

```
adjacencyList_1017 = new HashMap<>();
for (String v_1017 : vertices_1017) {
    adjacencyList_1017.put(v_1017, new ArrayList<>());
```

Membuat struktur adjacency list, yaitu struktur data yang menyimpan hubungan antar node dalam bentuk list.

```
addEdge_1017("Parkiran", "Pascasarjana");
addEdge_1017("Parkiran", "Lab Teknik Komputer");
addEdge_1017("Pascasarjana", "PKM");
addEdge_1017("Lab Teknik Komputer", "Dept Teknik Komputer");
addEdge_1017("Lab Sistem Informasi", "Seminar E");
addEdge_1017("Lab Sistem Informasi", "Dept Sistem Informasi");
addEdge_1017("PKM", "Lobby");
addEdge_1017("Dept Teknik Komputer", "Dept Sistem Informasi");
addEdge_1017("Dept Teknik Komputer", "Dekanat");
addEdge_1017("Dept Sistem Informasi", "Lab Informatika");
addEdge_1017("Dept Sistem Informasi", "Mushalla");
addEdge_1017("Lab Informatika", "Dept Informatika");
addEdge_1017("Lab Informatika", "Seminar E");
addEdge_1017("Lobby", "Dekanat");
addEdge_1017("Dekanat", "Mushalla");
addEdge_1017("Mushalla", "Dept Informatika");
```

Digunakan untuk menghubungkan node-node pada graf sehingga membentuk struktur jaringan. Setiap pemanggilan addEdge_1017() berarti menambahkan hubungan antar dua node.

```
private void addEdge_1017(String u_1017, String v_1017) {
    adjacencyList_1017.get(u_1017).add(v_1017);
    adjacencyList_1017.get(v_1017).add(u_1017);
}
```

Menambahkan edge (hubungan) antar dua node. Karena ditambahkan dua arah, maka graf bersifat tidak berarah (undirected graph).

```
private void initializeGUI_1017() {
    setLayout(new BorderLayout());

    JPanel controlPanel_1017 = new JPanel(new
FlowLayout(FlowLayout.CENTER, 10, 10));
    controlPanel_1017.setBackground(new Color(240, 240, 240));

    startComboBox_1017 = new JComboBox<>(vertices_1017.toArray(new
String[0]));
    goalComboBox_1017 = new JComboBox<>(vertices_1017.toArray(new
String[0]));
}
```

Membuat panel kontrol serta dropdown untuk memilih lokasi awal dan tujuan. Komponen ini memudahkan user memilih node. Tanpa ini, input harus dimasukkan secara manual.

```
JButton btnBFS_1017 = new JButton("BFS");
JButton btnDFS_1017 = new JButton("DFS");
JButton btnReset_1017 = new JButton("RESET");

btnBFS_1017.setBackground(new Color(70, 130, 180));
btnBFS_1017.setForeground(Color.WHITE);
btnDFS_1017.setBackground(new Color(34, 139, 34));
btnDFS_1017.setForeground(Color.WHITE);
btnReset_1017.setBackground(new Color(220, 20, 60));
btnReset_1017.setForeground(Color.WHITE);
```

Bagian ini membuat tombol untuk menjalankan BFS, DFS, dan reset. Tombol ini menjadi kontrol utama algoritma.

```
visualizationPanel_1017 = new JPanel() {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        drawGraph_1017(g);
    }
}
```

Digunakan untuk menggambar graf secara visual. Method paintComponent di-override agar bisa custom drawing. Panel ini menjadi area utama visualisasi graf.

```
private void drawGraph_1017(Graphics g) {
    Graphics2D g2d_1017 = (Graphics2D) g;
    g2d_1017.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

    g2d_1017.setColor(Color.LIGHT_GRAY);
    g2d_1017.setStroke(new BasicStroke(3));
    for (String u_1017 : adjacencyList_1017.keySet()) {
        Point p1_1017 = nodePositions_1017.get(u_1017);
        for (String v_1017 : adjacencyList_1017.get(u_1017)) {
            Point p2_1017 = nodePositions_1017.get(v_1017);
            g2d_1017.drawLine(p1_1017.x, p1_1017.y, p2_1017.x, p2_1017.y);
        }
    }
}
```


Menggambar semua edge pada graf. Setiap garis menunjukkan hubungan antar node. Ini memberikan gambaran struktur jaringan secara keseluruhan.

```
if (path_1017 != null && path_1017.size() > 1) {
    g2d_1017.setColor(Color.RED);
    g2d_1017.setStroke(new BasicStroke(5));
    for (int i = 0; i < path_1017.size() - 1; i++) {
        Point p1_1017 = nodePositions_1017.get(path_1017.get(i));
        Point p2_1017 = nodePositions_1017.get(path_1017.get(i+1));
        g2d_1017.drawLine(p1_1017.x, p1_1017.y, p2_1017.x, p2_1017.y);
    }
}
```

Menampilkan jalur hasil BFS/DFS dengan warna merah. Jalur ini menunjukkan rute dari start ke goal. Ini memudahkan user melihat hasil pencarian secara langsung.

```
for (String node_1017 : vertices_1017) {
    Point p_1017 = nodePositions_1017.get(node_1017);
}
```

Melakukan iterasi semua node agar bisa digambar. Setiap node akan diproses satu per satu. Ini memastikan semua titik tampil di layar.

```
if (path_1017 != null && path_1017.contains(node_1017)) {
    g2d_1017.setColor(new Color(255, 100, 100));
} else if (visitedNodes_1017 != null &&
visitedNodes_1017.contains(node_1017)) {
    g2d_1017.setColor(new Color(255, 220, 100));
} else {
    g2d_1017.setColor(new Color(173, 216, 230));
}
```

Kode ini digunakan untuk menentukan warna node:

- Merah : bagian dari path
- Kuning : sudah dikunjungi
- Biru : belum dikunjungi

```
public void BFS_1017() {
    String start_1017 = (String) startComboBox_1017.getSelectedItem();
    String goal_1017 = (String) goalComboBox_1017.getSelectedItem();

    if (start_1017.equals(goal_1017)) {
        resultArea_1017.setText("Lokasi awal dan tujuan sama!");
        return;
    }

    Queue<String> queue_1017 = new LinkedList<>();
    Map<String, String> parent_1017 = new HashMap<>();
    Set<String> visited_1017 = new LinkedHashSet<>();
}
```

Method BFS digunakan untuk mencari jalur menggunakan algoritma Breadth First Search. Pada bagian awal, program mengambil input node awal (start) dan node tujuan (goal) dari combobox. Jika start dan goal sama, maka proses dihentikan karena tidak perlu pencarian. Setelah itu dibuat struktur data queue, parent map, dan visited set untuk menyimpan proses BFS.

```
queue_1017.add(start_1017);
visited_1017.add(start_1017);
parent_1017.put(start_1017, null);

boolean found_1017 = false;
```

```

while(!queue_1017.isEmpty()) {
    String current_1017 = queue_1017.poll();
    if(current_1017.equals(goal_1017)) {
        found_1017 = true;
        break;
    }

    for(String neighbor_1017 : adjacencyList_1017.get(current_1017))
    {
        if(!visited_1017.contains(neighbor_1017)) {
            visited_1017.add(neighbor_1017);
            parent_1017.put(neighbor_1017, current_1017);
            queue_1017.add(neighbor_1017);
        }
    }
}

```

Node awal dimasukkan ke queue dan ditandai sebagai sudah dikunjungi. BFS bekerja dengan konsep FIFO (First In First Out), sehingga node yang masuk lebih dulu akan diproses lebih dulu. Setiap node yang diproses akan dicek apakah sudah mencapai tujuan. Jika belum, maka semua tetangga node tersebut akan dimasukkan ke queue jika belum pernah dikunjungi. Parent disimpan untuk membentuk jalur akhir.

```

visitedOrder_1017 = new ArrayList<>(visited_1017);
visitedNodes_1017 = visited_1017;
exploredCountLabel_1017.setText("Jumlah Node Dikunjungi: " +
visited_1017.size());
algorithmUsed_1017 = "BFS";

```

Program menyimpan urutan node yang dikunjungi dan jumlah node yang dieksplorasi. Label GUI juga diperbarui untuk menampilkan jumlah node yang telah dikunjungi. Algoritma yang digunakan dicatat sebagai BFS.

```

if(found_1017) {
    path_1017 = new ArrayList<>();
    String curr_1017 = goal_1017;
    while(curr_1017 != null) {
        path_1017.add(curr_1017);
        curr_1017 = parent_1017.get(curr_1017);
    }
    Collections.reverse(path_1017);
    displayPath_1017();
} else {
    resultArea_1017.setText("Jalur tidak ditemukan!");
    path_1017 = null;
}
displayGraph_1017();

```

Jika tujuan ditemukan, maka jalur dibentuk kembali dari goal ke start menggunakan parent map. Setelah itu jalur dibalik agar urut dari start ke goal. Jika tidak ditemukan, maka ditampilkan pesan bahwa jalur tidak ada. Setelah itu graf diperbarui pada GUI.

```

public void DFS_1017() {
    String start_1017 = (String) startComboBox_1017.getSelectedItem();
    String goal_1017 = (String) goalComboBox_1017.getSelectedItem();

    if (start_1017.equals(goal_1017)) {
        resultArea_1017.setText("Lokasi awal dan tujuan sama!");
        return;
    }
}

```

```

    }

    Stack<String> stack_1017 = new Stack<>();
    Map<String, String> parent_1017 = new HashMap<>();
    Set<String> visited_1017 = new LinkedHashSet<>();

```

Method DFS digunakan untuk mencari jalur menggunakan Depth First Search. Input start dan goal diambil dari GUI. Jika sama, proses dihentikan. DFS menggunakan stack (LIFO) untuk menyimpan node yang akan diproses. Parent dan visited juga digunakan seperti BFS.

```

stack_1017.push(start_1017);
visited_1017.add(start_1017);
parent_1017.put(start_1017, null);

boolean found_1017 = false;

while(!stack_1017.isEmpty()) {
    String current_1017 = stack_1017.pop();
    if(current_1017.equals(goal_1017)) {
        found_1017 = true;
        break;
    }

    for(String neighbor_1017 : adjacencyList_1017.get(current_1017))
    {
        if(!visited_1017.contains(neighbor_1017)) {
            visited_1017.add(neighbor_1017);
            parent_1017.put(neighbor_1017, current_1017);
            stack_1017.push(neighbor_1017);
        }
    }
}

```

DFS bekerja dengan konsep LIFO, sehingga node terakhir yang masuk akan diproses lebih dulu. Setiap node akan dieksplorasi sampai kedalaman maksimum sebelum kembali (backtracking). Tetangga node dimasukkan ke stack jika belum pernah dikunjungi. Parent disimpan untuk membentuk jalur.

```

visitedOrder_1017 = new ArrayList<>(visited_1017);
visitedNodes_1017 = visited_1017;
exploredCountLabel_1017.setText("Jumlah Node Dikunjungi: " +
visited_1017.size());
algorithmUsed_1017 = "DFS";

```

Program menyimpan urutan node yang dikunjungi dan jumlahnya. Informasi ini ditampilkan pada GUI agar user mengetahui hasil proses DFS.

```

if(found_1017) {
    path_1017 = new ArrayList<>();
    String curr_1017 = goal_1017;
    while(curr_1017 != null) {
        path_1017.add(curr_1017);
        curr_1017 = parent_1017.get(curr_1017);
    }
    Collections.reverse(path_1017);
    displayPath_1017();
} else {
    resultArea_1017.setText("Jalur tidak ditemukan!");
    path_1017 = null;
}

```

```

    }
    displayGraph_1017();

```

Jika tujuan ditemukan, jalur dibentuk kembali dari goal ke start menggunakan parent map. Jika tidak ditemukan, maka ditampilkan pesan error. Setelah itu graf diperbarui.

```

public void displayPath_1017() {
    StringBuilder sb_1017 = new StringBuilder();
    sb_1017.append("Algoritma:");
    sb_1017.append(algorithmUsed_1017).append("\n\n");
    sb_1017.append("Jalur:\n");
    if (path_1017 != null) {
        sb_1017.append(String.join(" -> ", path_1017)).append("\n\n");
    }
    sb_1017.append("Node Dikunjungi:\n");
    if (visitedOrder_1017 != null) {
        sb_1017.append(String.join(", ", visitedOrder_1017));
    }
    resultArea_1017.setText(sb_1017.toString());
}

```

Method ini digunakan untuk menampilkan hasil pencarian ke dalam GUI. Informasi yang ditampilkan meliputi algoritma yang digunakan, jalur yang ditemukan, serta node yang telah dikunjungi. Semua data digabung menggunakan StringBuilder agar lebih efisien.

```

public void resetGraph_1017() {
    path_1017 = null;
    visitedOrder_1017 = null;
    visitedNodes_1017 = null;
    algorithmUsed_1017 = "";
    exploredCountLabel_1017.setText("Jumlah Node Dikunjungi: 0");
    resultArea_1017.setText("");
    displayGraph_1017();
}

```

Method ini digunakan untuk mengembalikan semua variabel ke kondisi awal. Semua hasil pencarian dihapus agar user dapat melakukan pencarian baru. Label GUI juga direset menjadi 0.

```

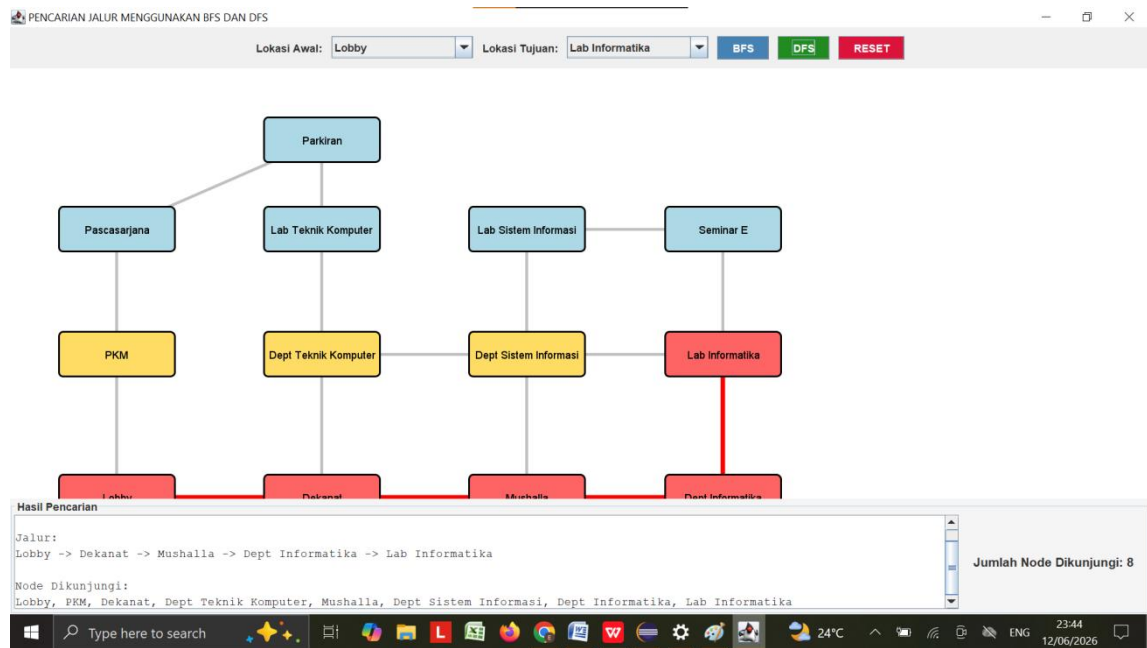
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new PetaFTIUnand_2511531017().setVisible(true);
    });
}

```

Program dijalankan dalam thread Swing agar GUI tampil dengan benar. Frame utama kemudian ditampilkan ke user.

Output:





Gambar 2.6

BAB III

KESIMPULAN

1.1 Kesimpulan

Struktur data Graf (Graph) sangat efektif dan fleksibel dalam memodelkan entitas dunia nyata yang saling terhubung secara kompleks. Kasus pemetaan lokasi kampus Universitas Andalas—seperti rektorat, kelas, dan laboratorium—berhasil direpresentasikan secara akurat menggunakan konsep vertex dan edge. Penerapan GUI menggunakan Java Swing mampu mentransformasikan representasi graf yang abstrak menjadi bentuk visual interaktif, sehingga mempermudah pengguna dalam melacak rute serta melihat perubahan status node secara langsung melalui indikator warna.

Fungsionalitas program juga sangat ditentukan oleh manajemen data internal dan penanganan multi-threading. Penggunaan fungsi penelusuran yang dinamis serta fungsi pembersihan seperti `resetGraph()` menggunakan `StringBuilder` terbukti vital dalam menjaga konsistensi data di dalam memori. Mekanisme pembersihan state ini memastikan bahwa aplikasi dapat melakukan pencarian jalur baru berulang kali secara tepat tanpa mengalami kebocoran memori (memory leak). Integrasi antara struktur graf, antarmuka grafis, dan kontrol data yang optimal berhasil menciptakan aplikasi simulasi rute yang efisien dan interaktif.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

- [1] Oracle, "Java Platform, Standard Edition 8 API Specification," Oracle Help Center, 2014. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/>.
- [2] Y. D. Liang, *Introduction to Java Programming and Data Structures, Comprehensive Version*, 12th ed. New York, NY, USA: Pearson, 2020.